

JCS2201 PYTHON PROGRAMMING

UNIT 1 INTRODUCTION TO PYTHON PROGRAMMING

I Year AI & ML – II Semester

UNIT 1 INTRODUCTION TO PYTHON PROGRAMMING SYLLABUS

- History of Python
- Features
- Installing Python
- Demo Of Interactive and script mode
- Identifiers
- Reserved Words
- Indentation
- Comments
- Variables
- Data Types and its Conversion
- Operators and its precedence
- Expressions
- Input and Print functions
- Command-Line arguments

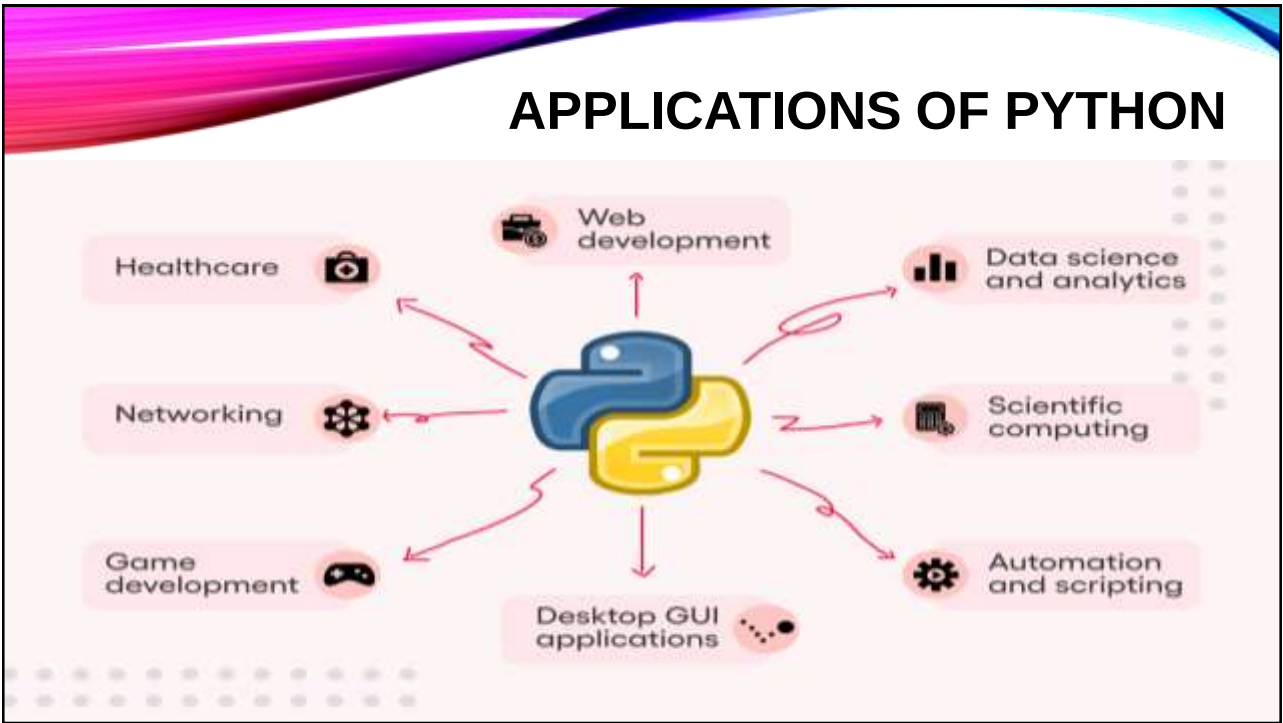


HISTORY OF PYTHON

- A widely used general-purpose, high-level programming language.
- Initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation.
- Was mainly developed for emphasis on code readability.
- Its syntax allows programmers to express concepts in fewer lines of code.
- He started the development of Python firstly as a hobby project because he was looking for an interesting project to keep him occupied during Christmas.
- The inspiration for the name came from BBC's TV Show – 'Monty Python's Flying Circus', as he was a big fan of the TV show.
- Also he wanted a short, unique and slightly mysterious name for his invention and hence he named it Python!

HISTORY OF PYTHON

- The language was finally released in 1991.
- When it was released, it used a lot fewer codes to express the concepts, when we compare it with Java, C++ & C. Its design philosophy was quite good too.
- Main objective - to provide code readability and advanced developer productivity.
- When it was released it had more than enough capability to provide classes with inheritance, several core data types exception handling and functions.
- Used for various purposes such as developing, scripting, generation and software testing.
- Elegant and simple – so, top technology organizations like Dropbox, Google, Quora, Mozilla, Hewlett-Packard, Qualcomm, IBM, and Cisco have implemented Python.
- Python 3.9. 0 is the newest major release of the Python programming language, and it contains many new features and optimizations.



INSTALLING PYTHON

<https://www.digitalocean.com/community/tutorials/install-python-windows-10>

DEMO OF INTERACTIVE AND SCRIPT MODE

Interactive mode

Where code is typed into the Python interpreter directly in the prompt, next to `>>>`. This is useful for trying out small snippets of code, or for testing things out as you're writing them.

`>>>` is called as Chevron symbol (or) prompt.

Advantages:

- Great for testing out commands and getting immediate feedback.
- Can also be used to quickly execute commands on a remote server.

Disadvantage:

- Can be difficult to automate tasks. For larger programs, the interactive mode is not suitable.
- Editing the program in interactive mode is a tedious task.

INTERACTIVE MODE



```
>>> print(5)
5
>>> print("HI")
HI
>>> a=5
>>> b=10
>>> print(a+b)
15
>>> print("Good" * 4)
GoodGoodGoodGood
>>>
```

DEMO OF INTERACTIVE AND SCRIPT MODE

Script mode

Where you write your code in a .py file and then run it with the python command. This is the most common way that people use Python because it lets you write and save your code so that you can use it again later.

Advantages:

- A great way to automate tasks and run commands on a remote server.
- Can also be used to create files that will execute certain commands when they are run.
- Can be very helpful in automating tasks or setting up a development environment.

Disadvantage:

- Difficult to troubleshoot.
- It can be difficult to write a script that does what you want it to do.
- Scripts can be slow because they have to execute all of their commands serially.

SCRIPT MODE

sample.py - C:\Users\HP\AppData\Local\Programs\Python\Python311\sample.py (3.11.5)

```
1 a=10
2 b=20
3 c=a+b
4 print(c)
5
```

IDENTIFIERS

- Python Identifier → the name given to identify a variable, function, class, module or other object.
- Identifier → name give to an entity.

Examples: count, max_value, student1

IDENTIFIERS

Rules for Writing Identifiers

- Python is case sensitive.
- Identifiers can be combination of uppercase and lowercase letters, digits or an underscore(_). So **myVariable**, **variable_1**, **variable_for_print** all are valid python identifiers.
- An Identifier can not start with digit. So while **variable1** is valid, **1variable** is not valid.
- We can't use special symbols like **!,#,@,%,\$** etc in our Identifier.
- Identifier can be of any length.

IDENTIFIERS

Conventions (optional) for Writing Identifiers

- Class names start with an uppercase letter. All other identifiers start with a lowercase letter.
- Starting an identifier with a single leading underscore indicates the identifier is private.
- If the identifier starts and ends with two underscores, that means the identifier is language-defined special name.
- While **c = 10** is valid, writing **count = 10** would make more sense and it would be easier to figure out what it does even when you look at your code after a long time.
- Multiple words can be separated using an underscore, for example **this_is_a_variable**.

RESERVED WORDS

- Python keywords are the words that are reserved.
- They cannot be used name of any entities like variables, classes and functions.
- They are used for defining the syntax and structures of Python language.

False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	from	or	
continue	global	pass	

INDENTATION

- Indentation refers to the spaces at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code.

Example:

```
if 5 > 2:
    print("Five is greater than two!")
    print("Hello")
```

COMMENTS

- Comments are short descriptions along with the code to increase its readability.
- Used to explain the basic logic behind why a particular line of code was written
- Python interpreter completely ignores comments in Python
- Comments start with a #.
- 2 types → single line comments, multi-line comments

COMMENTS

Examples:

```
# Python program to find the square of a number
```

```
print("Hi" * 5) # Display HI 5 times
```

```
# Python program  
# to display  
# multiplication table
```

```
''' Python program  
to display  
multiplication table '''
```

VARIABLES

- A variable indicates an entity in a Python program whose value is changeable over time.
- It is a memory location where a value can be stored. Later we can retrieve the value to use.
- Variable is the name given to the memory location that holds a value.
- Variables are case sensitive. Eg. variable **Age** is different from variable **age**.

Examples:

a = 5 → here 'a' is a variable
 count = 0 → here 'count' is a variable
 print(a) → here variable 'a' is used

VARIABLES

- Multiple variables can be assigned with same or different values simultaneously.

```
name = "Peter"
Name = "Ria"
print("My name is ", name)
print("My friend's name is ", Name)
```

```
a = b = c = 86
print(a)
print(b)
```

OUTPUT:

```
>>>
86
86
```

```
mat, phy, chem = 86, 92, 89
print(mat)
print(chem)
```

OUTPUT:

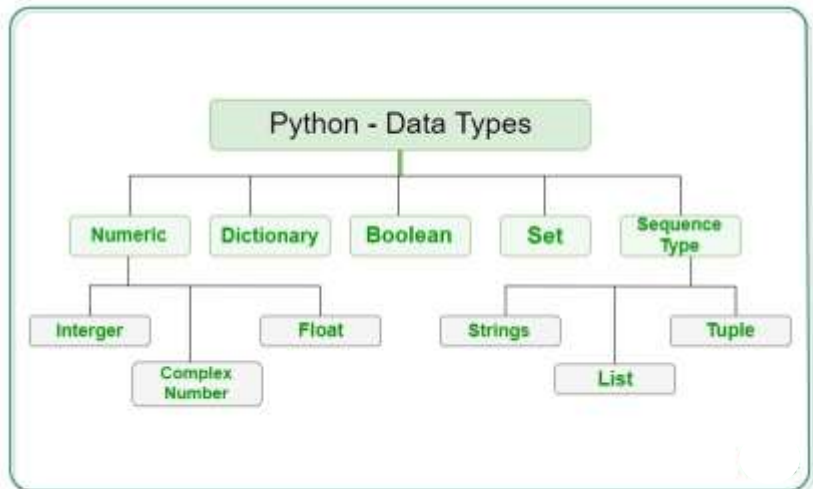
```
>>>
86
89
```

OUTPUT:

```
>>>
My name is Peter
My friend's name is Ria
```

DATA TYPES AND ITS CONVERSION

- Python has various standard data types.
- They are used to define the operations that can be performed on them and the storage method for each of them.



DATA TYPES AND ITS CONVERSION

- To get the data type of a variable – use `type( )` function.

```
>>> x = 5
>>> print(type(x))
```

DATA TYPES

1. NUMERIC DATA TYPE

- To store numeric values
- Python supports integers, floating point numbers and complex numbers.
- Operation on numbers – arithmetical operations(addition, subtraction, multiplication, division, modulo division) and comparison operations
- Eg. var1 = 10, temp = 98.4
- Python supports 4 numerical data types.

INTEGERS	LONG	FLOAT	COMPLEX
- Called as int - Can be positive or negative numbers with no decimal point	- Long integers - Can also be represented in Octal and Hexadecimal	- Numbers with decimal point - Has integer part and fractional part	- Numbers of the form $a + bj$, where a and b are int / float numbers and $j = \sqrt{-1}$ - Here a is real part and b is imaginary part
Eg. 100	Eg. 516762819L	Eg. 0.0, 15.856, -0.7, -32.54e100	Eg. 3 - 4j, 2.8 + 6.1j, 0.87j

DATA TYPES

2. DICTIONARY

- Dictionary is an unordered collection of sets of items, which appear as key-value pairs..
- Dictionary is created using curly brackets { }
- Dictionary items are accessed via keys and not by their positions
- Eg. dict1 = {"Apple" : 200, "Orange" : 150, "Kiwi" : 320}
- Here dict1[Apple] = 200

```
dict1 = {"Apple" : 200, "Orange" : 150, "Kiwi" : 320}
print(dict1["Apple"])
```

```
OUTPUT:
200
>>>
```

DATA TYPES

3. BOOLEAN

- Boolean represents the values – True or False
- True and False can be assigned as values for a variable
- True represents 1 and False represents 0.

```
x = True  
print(x)
```

```
OUTPUT:  
True  
>>>
```

DATA TYPES

4. SET

- Sets are used to store multiple items in a single variable.
- A set is a collection which is unordered, unchangeable, and unindexed.
- Set items are unchangeable, but you can remove items and add new items.
- Set items are unordered, unchangeable, and do not allow duplicate values.
- Set items can appear in a different order every time you use them, and cannot be referred to by index or key.
- Eg. set1 = {"apple", "banana", "cherry"}

```
set1 = {"apple", "banana", "cherry"}  
print(set1)
```

```
OUTPUT:  
{"apple", "banana", "cherry"}  
>>>
```

DATA TYPES

4. SET examples

```
set1 = {"apple", "banana", "cherry"}
print(set1)
```

```
OUTPUT:
{'banana', 'cherry', 'apple'}
>>>
```

```
set1 = {"apple", "banana", "cherry", "apple"}
print(set1)
```

```
OUTPUT:
{'banana', 'cherry', 'apple'}
>>>
```

```
set1 = {"apple", "banana", "cherry", True, 1, 2}
print(set1)
```

```
OUTPUT:
{True, 2, 'banana', 'cherry', 'apple'}
>>>
```

DATA TYPES

5. SEQUENCE TYPE

- Sequence is an ordered collection of items, indexed by positive integers.
- 3 types – 1. Strings 2. Lists 3. Tuples

STRINGS	LISTS	TUPLES
- String is a sequence (or) collection of characters like numbers, letters and special characters (symbols)	- List is an ordered sequence of items - It is mutable → values in a list can be changed - Values in the list are called elements / items.	- Tuple is an ordered sequence of items - It is an immutable list → values in a tuple cannot be changed or removed
- Marked by single quotes ' ' or double quotes " " or triple quotes """ """	- It can be written as a list of comma-separated items (values) between square brackets [].	- It can be written as a list of comma-separated items (values) enclosed in parentheses ().
- Individual character is indexed and accessed using index.	- Items in the lists can be of different data types.	- Faster than lists - Data to be protected from changes can be stored in tuples
Eg. a = "APPLE" Here a[0] = 'A', a[3] = 'L'	Eg. list1=["apple", 26, 3.5, "orange"] print(list1[0]) → apple list1[2] = 5.8 → changes 3.5 into 5.8	Eg. tup1= ("apple", 26, 3.5, "orange") print(tup1[0]) → apple tup1[2] = 5.8 → leads to error

DATA TYPES

5. SEQUENCE TYPE - Strings

- String is a sequence (or) collection of characters like numbers, letters and special characters (symbols)
- Marked by single quotes ' ' or double quotes " " or triple quotes `""" """`
- Individual character is indexed and accessed using index.
- Eg. `a = "APPLE"` → Here `a[0] = 'A'`, `a[3] = 'L'`

DATA TYPES

5. SEQUENCE TYPE - List

- List is an ordered sequence of items
- It is mutable → values in a list can be changed
- Values in the list are called elements / items.
- It can be written as a list of comma-separated items (values) between square brackets [].
- Items in the lists can be of different data types.
- Eg.

```
list1= ("apple", 26, 3.5, "orange")
print(list1[0])
```

OUTPUT:
apple
>>>

- `list1[2] = 5.8` → changes 3.5 into 5.8

DATA TYPES

5. SEQUENCE TYPE - Tuple

- Tuple is an ordered sequence of items
- It is an immutable list → values in a tuple cannot be changed or removed
- It can be written as a list of comma-separated items (values) enclosed in parentheses ().
- Faster than lists
- Data to be protected from changes can be stored in a tuple

```
tup1= ("apple", 26, 3.5, "orange")  
print(tup1[0])  
tup1[2] = 5.8
```

```
OUTPUT:  
apple  
>>> error statement
```

CONVERSIONS OF DATA TYPES

Type conversion is the process of converting data of one type to another.

Two types of type conversion in Python.

Implicit type conversion - automatic type conversion

Explicit type conversion - manual type conversion

CONVERSIONS OF DATA TYPES

Implicit type conversion

- Automatic type conversion
- In certain situations, Python automatically converts one data type to another. This is known as implicit type conversion.
- Python always converts smaller data types to larger data types to avoid the loss of data → converts integer to float.

```
a = 123
b = 1.23
c = a + b
```

```
print("Sum = ", c)
print("Data Type of sum = ", type(c))
```

OUTPUT:

```
Sum = 124.23
```

```
Data Type of sum = <class 'float'>
```

```
>>>
```

CONVERSIONS OF DATA TYPES

Explicit type conversion

- Users convert the data type of an object to required data type, manually using the built-in functions like `int( )`, `float( )`, `str( )`. This is known as explicit type conversion.
- Also called as typecasting.

Functions	Purpose
<code>int()</code>	converts any data type to an integer
<code>float()</code>	converts any data type to a floating-point number
<code>str()</code>	converts an integer into a string
<code>complex(real,imag)</code>	converts real numbers to complex(real,imag) number.

CONVERSIONS OF DATA TYPES

```
a = '12'  
b = 23  
c = a + b  
print("Sum = ", c)  
print("Data type of sum = ",type(c))
```

OUTPUT:
ERROR!
TypeError: can only concatenate str (not "int") to str
>>>

```
a = '12'  
b = 23  
a = int(a)  
c = a + b  
print("Sum = ", c)  
print("Data type of sum = ",type(c))
```

OUTPUT:
Sum = 35
Data type of sum = <class 'int'>
>>>

CONVERSIONS OF DATA TYPES

Sl.No.	Functions	Purpose
1	int()	converts any data type to an integer
2	float()	converts any data type to a floating-point number
3	str()	converts an integer into a string
4	complex()	converts real numbers to complex(real,imag) number
5	bool()	converts a value to a Boolean value
6	ord()	converts a character to an integer
7	hex()	converts an integer to a hexadecimal string
8	oct()	converts an integer to an octal string
9	tuple()	converts a sequence type to a tuple.
10	set()	converts a sequence type to a set type
11	list()	converts a sequence type to a list type
12	chr()	converts a number to its corresponding ASCII character

CONVERSIONS OF DATA TYPES

```
s = "10010"  
c = int(s)  
print ("After converting to integer : ", c)  
d = int(s,2)  
print ("After converting to integer base 2 : ", d)  
e = float(s)  
print ("After converting to float : ", e)  
g = str(c)  
print ("After converting integer to string : ", g)
```

OUTPUT:

```
After converting to integer : 10010  
After converting to integer base 2 : 18  
After converting to float : 10010.0  
After converting integer to string : 10010
```

CONVERSIONS OF DATA TYPES

```
a = 10  
b = 20  
c = complex(a, b)  
print ("After converting integer to complex number : ", c)
```

OUTPUT:

```
After converting integer to complex number : 10+20j
```

CONVERSIONS OF DATA TYPES

- The Boolean values will be returned as a result of some sort of comparison when `bool( )` function is used.
- Numbers can be used as bool values by using Python's built-in `bool()` method.
- Any integer, floating-point number, or complex number having zero as a value is considered as False, while if they are having value as any positive or negative number then it is considered as True.

```
a = 10
b = 20
c = 0
d, e = 30, -56
print (bool(a<b))
print(bool(c))
print(bool(d), bool(e))
```

OUTPUT:
True
False
True True

CONVERSIONS OF DATA TYPES

```
s = '4'
c = ord(s)
print ("After converting character to integer : ",c)
d = hex(56)
print ("After converting 56 to hexadecimal string : ",d)
e = oct(56)
print ("After converting 56 to octal string : ",e)
```

OUTPUT:

After converting character to integer : 52
After converting 56 to hexadecimal string : 0x38
After converting 56 to octal string : 0o70

CONVERSIONS OF DATA TYPES

```
s = 'Python'
c = tuple(s)
print ("After converting string to tuple : ", c)
d = set(s)
print ("After converting string to set : ", d)
e = list(s)
print ("After converting string to list : ",e)
```

OUTPUT:

```
After converting string to tuple : ('P', 'y', 't', 'h', 'o', 'n')
After converting string to set : {'P', 'o', 'h', 'y', 't', 'n'}
After converting string to list : ['P', 'y', 't', 'h', 'o', 'n']
```

CONVERSIONS OF DATA TYPES

```
a = chr(76)
b = chr(77)
print(a)
print(b)

c = 'L'
print(ord(c))
```

OUTPUT:

```
L
M
76
```

INPUT AND PRINT FUNCTIONS

Input function used → `input( )`

Output function used → `print( )`

To get user input with a message

```
name = input("Enter your name: ")
print("Hello, " + name)
print(type(name))
```

OUTPUT:

```
Enter your name: Suresh
Hello, Suresh
<class 'str'>
```

To get integer input with a message

```
count = int(input("Enter the count: "))
print("COUNT = ", count)
print("INCREASED COUNT = ", count + 1)
```

OUTPUT:

```
Enter the count: 27
COUNT = 27
INCREASED COUNT = 28
```

INPUT AND PRINT FUNCTIONS

To get multiple inputs → using `map( )` and `split( )` methods

```
a, b, c = map(int, input("Enter 3 Numbers : ").split())
print("The numbers are : ", a, b, c)
```

OUTPUT:

```
Enter 3 Numbers : 2 9 5
The numbers are : 2 9 5
```

INPUT AND PRINT FUNCTIONS

Input function used → `input( )`

Output function used → `print( )`

To display output

```
print("PYTHON")
print('P', 'Y', 'T', 'H', 'O', 'N')
```

OUTPUT:
PYTHON
P Y T H O N

To display output with sep and end parameters

```
print("PYTHON", end = '*')
print('P', 'Y', 'T', 'H', 'O', 'N', sep = '$')
```

OUTPUT:
PYTHON*P\$Y\$T\$H\$O\$N

INPUT AND PRINT FUNCTIONS

To display formatted output - String formatting using F string

```
name = "Python"
print(f'Hello {name}! How are you?')
```

OUTPUT:
Hello Python! How are you?

Formatting using `format()` function

```
a = 20
b = 10
sum = a + b
print('The value of a is {} and b is {}'.format(a,b))
print('{2} is the sum of {0} and {1}'.format(a,b,sum))
```

OUTPUT:
The value of a is 20 and b is 10
30 is the sum of 20 and 10

INPUT AND PRINT FUNCTIONS

To display formatted output – using % operator

```
P = 10000
```

```
N = 2
```

```
R = 2.5
```

```
si = P * N * R / 100
```

```
print('The value of simple interest is %d' %si)
```

OUTPUT:

The value of simple interest is 500

To display formatted output – print 2 decimal places

```
P, N, R = 10845, 2, 2.5
```

```
si = P * N * R / 100
```

```
print('The value of simple interest is Rs.{:0.2f}'.format(si))
```

OUTPUT:

The value of simple interest is Rs.498.87

COMMAND-LINE ARGUMENTS

- The arguments that are given after the name of the program in the command line shell of the operating system are known as Command Line Arguments.
- Python provides various ways of dealing with these types of arguments.
 - Using sys.argv
 - Using getopt module
 - Using argparse module

COMMAND-LINE ARGUMENTS

Using sys.argv

- The sys module provides functions and variables used to manipulate different parts of the Python runtime environment.
- This module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter.
- One such variable is sys.argv which is a simple list structure.
- The main purpose of sys.argv:
 - It is a list of command line arguments.
 - len(sys.argv) provides the number of command line arguments.
 - sys.argv[0] is the name of the current Python script.

COMMAND-LINE ARGUMENTS

Example program to add 2 numbers passed as Command-Line arguments

```
import sys
n = len(sys.argv)
print("Total arguments passed:", n)
print("\nName of the Python script:", sys.argv[0])
sum = int(sys.argv[1]) + int(sys.argv[2])
print("\nSum of the 2 values = ", sum)
```

OUTPUT:

```
C:\Users\HP\AppData\Local\Programs\Python\Python311>python sample.py 1 2
Total arguments passed: 3
Name of the Python script: sample.py
Sum of the 2 values = 3
```

COMMAND-LINE ARGUMENTS

Example program to swap 2 numbers passed as Command-Line arguments

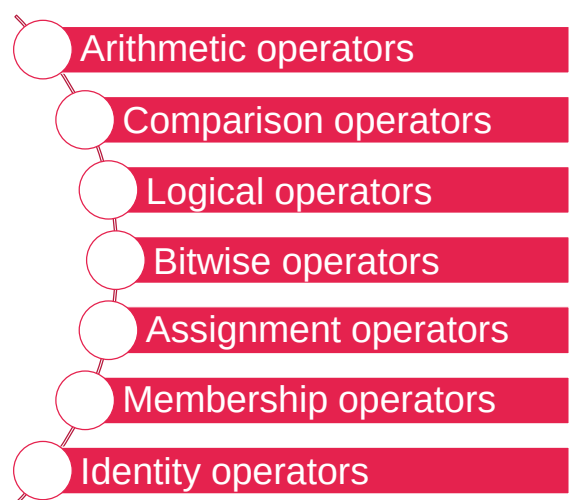
```
import sys
a = int(sys.argv[1])
b = int(sys.argv[2])
print("\nValues before swapping: a = {}, b = {}".format(a,b))
a,b = b, a
print("\nValues after swapping: a = {}, b = {}".format(a,b))
```

OUTPUT:

```
C:\Users\HP\AppData\Local\Programs\Python\Python311>python sample.py 10 20
Values before swapping: a = 10, b = 20
Values after swapping: a = 20, b = 10
```

OPERATORS

- Operators – symbols used to perform arithmetic and logical manipulations on variables and values.
- Python divides the operators in the following groups:



1. ARITHMETIC OPERATORS

- They are used to perform arithmetic operations on numeric data.

Sl. No.	OPERATORS	DESCRIPTION	EXAMPLE (Let x = 25, y = 3)
1	+	Addition	$x + y \rightarrow 28$
2	-	Subtraction	$x - y \rightarrow 22$
3	*	Multiplication	$x * y \rightarrow 75$
4	/	Division	$x / y \rightarrow 8.666667$
5	%	Modulus division	$x \% y \rightarrow 1$
6	//	Floor division	$x // y \rightarrow 8$
7	**	Exponentiation	$x ** y \rightarrow 25^3 \rightarrow 15625$

1. ARITHMETIC OPERATORS

- Modulus division operator % gives the remainder after a division.
- Eg. $39 \% 5 = 4$
- Floor division operator // rounds off the result down to the nearest lower integer (whole number)
- Eg. $39 / 7 = 5.571428571$ But $39 // 7 = 5$

2. COMPARISON OPERATORS

- They are used to compare two values or variables. Returns True or False

Sl. No.	OPERATORS	DESCRIPTION	EXAMPLE (Let x = 5, y = 3, z = 5)
1	==	Equal	x == y → False
2	!=	Not equal	x != y → True
3	>	Greater than	x > y → True
4	<	Less than	x < y → False
5	>=	Greater than or equal to	x >= z → True
6	<=	Less than or equal to	x <= y → False

3. LOGICAL OPERATORS

- They are used to combine conditional statements.

Sl. No.	OPERATORS	DESCRIPTION	EXAMPLE (Let x = 5, y = 3)
1	and	Returns True if both conditions are true	x > y and x < 10 → True x <= y and y > 0 → False
2	or	Returns True if one or both of the conditions are true	x != y or x < y → True x == 0 or y == 0 → False
3	not	Reverse the result - returns False if the condition is true	not (x < 0) → True

4. BITWISE OPERATORS

- They are used to compare binary numbers bit-by-bit.

Sl. No.	OPERATORS	NAME	DESCRIPTION	EXAMPLE (Let x = 6, y = 3)
1	&	Bitwise AND	Sets each bit to 1 if both bits are 1	$x \& y \rightarrow 2$
2		Bitwise OR	Sets each bit to 1 if both or one of two bits is 1	$x y \rightarrow 7$
3	^	Bitwise XOR	Sets each bit to 1 if only one of two bits is 1	$x \wedge y \rightarrow 5$
4	~	Bitwise NOT	Inverts all the bits – sets 1 to 0 and vice versa	$\sim y \rightarrow 253$
5	<<	Left shift operator	Shift bits to left from the right by some position	$x \ll y \rightarrow 48$
6	>>	Right shift operator	Shift bits to right from the left by some position	$x \gg 2 \rightarrow 1$

4. BITWISE OPERATORS

- If $x = 6$ and $y = 3$, then $x \& y$ can be calculated as below:

$x = 6 = 0000\ 0110$

$y = 3 = 0000\ 0011$

$x \& y = 0000\ 0010 \rightarrow 2$

- If $x = 6$ and $y = 3$, then $x \wedge y$ can be calculated as below:

$x = 6 = 0000\ 0110$

$y = 3 = 0000\ 0011$

$x \wedge y = 0000\ 0101 \rightarrow 5$

5. ASSIGNMENT OPERATORS

- They are used to assign values to variables.

SI. No.	OPERATORS	DESCRIPTION	EXAMPLE (Let x = 25, y = 3)
1	=	Assignment (equals)	x = 25 x = 25 + y x, y = 25, 3
2	+=	Addition assignment	x += y → x = x+y → x gets 28
3	-=	Subtraction assignment	x -= y → x = x-y → x gets 22
4	*=	Multiplication assignment	x *= y → x = x*y → x gets 75
5	/=	Division assignment	x /= y → x = x/y → x gets 8.666667
6	%=	Modulus division assignment	x %= y → x = x%y → x gets 1

5. ASSIGNMENT OPERATORS

- They are used to assign values to variables.

SI. No.	OPERATORS	DESCRIPTION	EXAMPLE (Let x = 25, y = 3)
7	//=	Floor division assignment	x //= y → x=x//y → x gets 8
8	**=	Exponentiation assignment	x **= y → x=x ** y → x gets 15625 (25 ³)
9	&=	Bitwise AND assignment	x &= y → x=x&y → x gets 1
10	=	Bitwise OR assignment	x = y → x=x y → x gets 27
11	^=	Bitwise XOR assignment	x ^= y → x=x^y → x gets 26
12	<<=	Left shift assignment	x <<= y → x=x<<y → x gets 200
13	>>=	Right shift assignment	x >>= y → x=x>>y → x gets 3

6. MEMBERSHIP OPERATORS

- They are used to check if a value or a variable is in a sequence, such as strings, lists, or tuples.

Sl. No.	OPERATORS	DESCRIPTION
1	in	Returns True if the specified element is present in a sequence; otherwise returns False
2	not in	Returns True if the specified element is not present in a sequence; otherwise returns False

6. MEMBERSHIP OPERATORS

• Examples:

```
x = 'apple'
print('a' in x)
print('L' in x)
print('M' in 'mango')
print('$' not in x)
```

OUTPUT:
True
False
False
True

```
fruits_list = ['apple', 'mango', 'guava', 'papaya'] # a list of fruits
print('apple' in fruits_list)
print('Mango' in fruits_list)
print('Mango' not in fruits_list)
```

OUTPUT:
True
False
True

```
fruits_dict = {1:'apple', 2:'mango', 8:'guava', 9:'papaya'}
print(3 in fruits_dict)
```

OUTPUT:
True

7. IDENTITY OPERATORS

- They are used to compare the objects if both the objects are actually of the same data type and share the same memory location.

Sl. No.	OPERATORS	DESCRIPTION
1	is	Returns True if both the variables point to the same object; returns false otherwise.
2	is not	Returns True if both variables are not the same object; returns false otherwise

7. IDENTITY OPERATORS

• Examples:

```
x = 'apple'
y = 'apple'
z = x
print(x is y)
print(x is not z)
```

OUTPUT:

True
False

```
x = 13
y = 13
z = x
print(x is y)
print(x is not z)
```

OUTPUT:

True
False

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)
print(a is c)
print(a is not c)
print(b is not a)
```

OUTPUT:

True
False
True
False

OPERATOR PRECEDENCE

- Operator precedence describes the order in which operations are performed.
- The precedence order is described in the table, starting with the highest precedence at the top.

Operator	Description	
()	Parentheses (grouping)	Highest Lowest
**	Exponentiation	
~x	Bitwise nor	
+x,-x	Positive , negative (unary +,-)	
*,/,//,%	Multiplication , division , floor division , remainder	
+, -	Addition , subtraction	
&	Bitwise and	
^	Bitwise XOR	
	Bitwise OR	
<,<=,>,>=,<=,!=,==,is,isnot	Comparisons (Relational operators),identity operators	
not x	Boolean NOT	
And	Boolean AND	
or	Boolean OR	

EXPRESSIONS

- Expression → it is a combination of values, variables, and operators that evaluates to a single value.
- An expression is evaluated as per the precedence of its operators.
- Types of expressions:
 1. Constant Expressions
 2. Arithmetic Expressions
 3. Integral Expressions
 4. Floating Expressions
 5. Relational Expressions
 6. Logical Expressions
 7. Bitwise Expressions
 8. Combinational Expressions

EXPRESSIONS

1. Constant Expressions: These are the expressions that have constant values only.

```
a = 37 + 8.2
print(a)
```

```
OUTPUT:
45.2
```

```
p = 40
q = 12
add = x + y
sub = x - y
pro = x * y
div = x / y
```

```
print(add)
print(sub)
print(pro)
print(div)
print((6 + 3) - (6 + 3))
print(100 + 5 * 3)
```

```
OUTPUT:
52
28
480
3.3333
0
115
```

2. Arithmetic Expressions: An arithmetic expression is a combination of numeric values, operators, and sometimes parenthesis. The result of this type of expression is also a numeric value.

EXPRESSIONS

3. Integral Expressions: This type of expression always produces an integer value as a result.

```
a = 37
b = 8.2
c = a + int(b)
print("Sum = ", c)
```

```
OUTPUT:
Sum = 45
```

4. Floating Expressions: This type of expression always produces a floating point number as a result.

```
a = 37
b = 8.2
c = float(a) + b
print("Sum = ", c)
```

```
OUTPUT:
Sum = 45.2
```

EXPRESSIONS

5. Relational Expressions: Relational expression is a combination of two or more arithmetic expressions joined using relational operators. The overall expression results in either True or False

```
a = 26
b = 14
c = (a + b) == (a * 2)
print('Result = ', c)
print(type(c))
```

OUTPUT:
Result = False
<class 'bool'>

6. Logical Expressions: Logical expression performs the logical computation, and the overall expression results in either True or False.

```
phy, che, mat = 66, 56, 86
result = (phy > 60) and (che > 60) or (mat > 80)
print('Result = ', result)
print(type(result))
```

OUTPUT:
Result = True
<class 'bool'>

EXPRESSIONS

7. Bitwise Expressions: Bitwise expression is used for operation or computation performed at the bit level.

```
x = 25
a = x << 1
b = x >> 1
print(a)
print(b)
```

OUTPUT:
50
12

8. Combinational Expressions: It can contain a single or multiple expressions which result in an integer or boolean value depending upon the expressions involved.

```
x = 25
y = 35
z = x * 2 + (y << 1)
print(f"RESULT = {z}")
```

OUTPUT:
Result = 120